

Assignment 1

1. What is Class and Object? How to create an Object?

- Class = blueprint/design
- Object = real thing made from class
Example:
- Class = Car
- Object = BMW, Audi

Car c = new Car();

2. What is OOP? Explain Java features?

OOP means Object Oriented Programming.
Everything works using objects.

Features:

- Class
- Object
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction

Example:

Like mobile apps use objects.

3. What are Java Data Types?

Used to store data.

Examples:

- int → numbers
- double → decimal
- char → single letter
- String → text

- boolean → true/false

Example:

```
int age = 20;
```

4. What is an Array?

Array stores multiple values in one variable.

Example:

Marks of students.

```
int arr[] = {10,20,30};
```

5. What is the role of constructor? Why use multiple constructors?

Constructor initializes object.

Multiple constructors give different ways to create objects.

Example:

- Product with default values
- Product with user values

6. Explain constructor overloading.

Same constructor name with different parameters.

Example:

```
Product()
```

```
Product(int id,String name)
```

7. Difference between instance variables and local variables.

- Instance variable → inside class

- Local variable → inside method

Example:

Balance in ATM = instance variable

Amount inside withdraw() = local variable

8. Why is Scanner used?

Scanner is used to take user input.

Example:

User enters name or marks.

9. Why use methods instead of writing everything in main()?

Methods make code simple and reusable.

Example:

Separate method for deposit and withdraw.

10. How does this system demonstrate Encapsulation?

Data and methods are inside one class.

Example:

ATM balance and methods inside ATM class.

11. Importance of this keyword.

`this` refers to current object.

Used to differentiate variables.

Example:

`this.id = id;`

12. Use of if, if-else, while, do-while, for loop.

- if → single condition
- if-else → two conditions
- while → repeats until condition false
- do-while → runs at least once
- for → fixed repetitions

Daily Example:

For loop = attendance of 60 students.

Assignment 2

1. What is method overloading in Java?

Same method name with different parameters.

Example:

```
power(int,int)
power(double,int)
```

2. Rules of method overloading.

- Same method name
 - Different parameters
-

3. Can overloading be done only by changing return type?

No.

Parameters must change.

4. Difference between overloading and overriding.

- Overloading → same class

- Overriding → child class changes parent method
-

5. What is a static method?

Method shared by all objects.

Example:

`Math.pow()`

6. What is a static variable?

Common variable shared by all objects.

Example:

Total books in library.

7. How we call static method in same class?

Directly by method name.

8. How we call static method of another class?

Using class name.

`Math.abs()`

9. Functions in Math class.

- `Math.pow()` → power
 - `Math.abs()` → absolute
 - `Math.sqrt()` → square root
 - `Math.max()` → maximum
-

10. Why Math.pow() called using class name?

Because it is static.

11. Package of Math class.

`java.lang`

12. Why Math class is not instantiated?

Because all methods are static.

13. What is compile-time polymorphism?

Method decided during compilation.

Example:

Method overloading.

14. Is method overloading polymorphism?

Yes.

It is compile-time polymorphism.

Assignment 3

1. What is static field, static class and static method?

- Static field → common variable
- Static method → common method
- Static nested class → class inside class

Example:

Total books.

2. Explain class and object.

Class = design

Object = real item

Example:

Class = Student

Object = Sujal

3. Why Java is called Object Oriented language?

Because Java uses classes and objects.

4. What is OOP? Features?

OOP = Object Oriented Programming

Features:

- Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction
-

5. What did you learn from this assignment?

- Static variable
 - Static method
 - Object creation
 - Library management logic
-

6. Difference between static and non-static method.

- Static → called using class name
- Non-static → needs object

7. How issuing and returning books handled?

Using methods:

- `issueBook()`
 - `returnBook()`
-

8. What is encapsulation?

Wrapping data and methods together.

Example:

Book details inside Book class.

Assignment 4

1. What is an array?

Stores multiple values.

Example:

Student marks.

2. Why use arrays?

To store many values together.

3. How declare array?

```
int arr[];
```

4. How initialize array?


```
int arr[] = {1,2,3};
```

5. How access array elements?

Using index.

```
arr[0]
```

6. Syntax for integer array.

```
int arr[] = new int[5];
```

7. How find size of array?

Using `.length`

8. Use of array.length?

Gives total elements count.

9. Can array size change?

No.

Assignment 5

1. Keyword used for inheritance?

`extends`

2. How subclass access superclass methods?

Using child object.

3. Can subclass use superclass methods?

Yes.

4. Can subclass override methods?

Yes.

5. What is method overriding?

Child class changes parent method.

6. Role of super keyword?

Calls parent class members.

7. How call superclass method?

Using `super`

8. Order of constructor execution?

Parent constructor first, then child.

9. Can subclass object call superclass methods?

Yes.

Example:
Dog object calling eat().

Assignment 6

What is interface?

Blueprint containing abstract methods.

Why interfaces used?

For multiple inheritance.

What is polymorphism?

One object many forms.

Example:

Animal a = new Dog();

Can class implement multiple interfaces?

Yes.

Difference between class and interface.

- Class has complete methods
 - Interface has abstract methods
-

Assignment 7

1. What is abstract class?

Class declared using `abstract`.

2. Can we create object of abstract class?

No.

3. Why use abstract class?

To provide common design.

4. Can abstract class have constructor?

Yes.

5. Difference between abstract class and interface.

- Abstract class can have normal methods
 - Interface mostly contains abstract methods
-

6. What happens if we instantiate abstract class?

Compile-time error.

7. Can abstract class implement interface?

Yes.

8. Can abstract class contain normal methods?

Yes.

Example:
display() method.

Assignment 8

1. What is OOP in Java?

Programming using classes and objects.

2. Main OOP concepts in ATM?

- Class
 - Object
 - Encapsulation
-

3. Main functions of ATM?

- Deposit
 - Withdraw
 - Check balance
-

4. How deposit() works?

Adds amount to balance.

5. How withdraw() checks insufficient balance?

Checks:

if(amount > balance)

6. What happens when amount greater than balance?

ArithmeticException occurs.

7. How balance displayed?

Using checkBalance() method.

8. What is exception?

Error during execution.

Example:
Divide by zero.

9. Difference between checked and unchecked exception.

- Checked → checked by compiler
 - Unchecked → runtime errors
-

10. Why try-catch used?

To handle errors without stopping program.

11. Exception for insufficient balance?

ArithmeticException

12. Exception for invalid input?

IllegalArgumentException

13. Role of finally block?

Always executes.

Example:

Thank You